Python: without numpy or sklearn

Q1: Given two matrices please print the product of those two matrices

```
Ex 1: A   = [[1 3 4]
             [2 5 7]
             [5 9 6]]
      B   = [[1 0 0]
             [0 1 0]
             [0 0 1]]
      A*B = [[1 3 4]
             [2 5 7]
             [5 9 6]]


Ex 2: A   = [[1 2]
             [3 4]]
      B   = [[1 2 3 4 5]
             [5 6 7 8 9]]
      A*B = [[11 14 17 20 23]
             [18 24 30 36 42]]

Ex 3: A   = [[1 2]
             [3 4]]
      B   = [[1 4]
             [5 6]
             [7 8]
             [9 6]]
      A*B =Not possible
```

In [44]:
```
# write your python code here
# you can take the above example as sample input for your program to tes
t
# it should work for any general input try not to hard code for only giv
en input examples


# you can free to change all these codes/structure
# here A and B are list of lists

def display(R):
    """
    Function display
    This function is used to print given matrix in user view format.

    :param R:
      A R that contains Matrix.

    :return: None
    """
    for i in range(len(R)):
        for j in range(len(R[0])):
            print("{}\t".format(R[i][j]), end=' ')
        print("\n")


def build(row, col, M):
    """
    Function create
    This function is used to create matrix from given rows and columns.

    :param row:
      A row that contains number of row of the Matrix.
    :param col:
      A col that contains number of column of the Matrix.
    :param M:
      A M that contain empty list.

    :return: None
    """
    for i in range(row):
        m = []
        for j in range(col):
            print("Enter matrix element: ")
            m.append(int(input()))
        M.append(m)

def matrix_mul(A, B):
    """
    Function matrix_mul
    This function is used to multiply two Matrix.

    :param A:
      A `A` that contains First Matrix.
    :param B:
      A `B` that contains Second Matrix.
```

```
    :return:
      Return resultant Matrix or None if Matrix multiplication is not po
ssible.
    """
    if len(A[0]) != len(B):
        print("Matrix Result is: ")
        print("Not possible!")
        return

    mul = [[0 for i in range(len(B[0]))] for j in range(len(A))]
    # write your code
    for k in range(len(A)):
        for i in range(len(B[0])):
            mul[k][i] = 0
            for j in range(len(B)):
                mul[k][i] += A[k][j] * B[j][i] # Multiplication

    return mul

A, B = [], []
print("Enter matrix A number of rows: ")
row = int(input())
print("Enter matrix A number of cols: ")
col = int(input())
build(row, col, A)
print("Enter matrix B number of rows: ")
row = int(input())
print("Enter matrix B number of cols: ")
col = int(input())
build(row, col, B)
print("Matrix A is: {}X{}\n".format(len(A), len(A[0])))

for i in range(len(A)):
    for j in range(len(A[0])):
        print("{:4}".format(A[i][j]), end=' ')
    print("\n")
print("Matrix B is: {}X{}\n".format(len(B), len(B[0])))

for i in range(len(B)):
    for j in range(len(B[0])):
        print("{:4}".format(B[i][j]), end=' ')
    print("\n")
R = matrix_mul(A, B)
if R:
    print("Matrix Result is: {}X{}\n".format(len(R), len(R[0])))

    for i in range(len(R)):
        for j in range(len(R[0])):
            print("{:4}".format(R[i][j]), end=' ')
        print("\n")
```

```
Enter matrix A number of rows:
3
Enter matrix A number of cols:
3
Enter matrix element:
1
Enter matrix element:
3
Enter matrix element:
4
Enter matrix element:
2
Enter matrix element:
5
Enter matrix element:
7
Enter matrix element:
5
Enter matrix element:
9
Enter matrix element:
6
Enter matrix B number of rows:
3
Enter matrix B number of cols:
3
Enter matrix element:
1
Enter matrix element:
0
Enter matrix element:
0
Enter matrix element:
0
Enter matrix element:
1
Enter matrix element:
0
Enter matrix element:
0
Enter matrix element:
0
Enter matrix element:
1
Matrix A is: 3X3

   1    3    4

   2    5    7

   5    9    6

Matrix B is: 3X3

   1    0    0

   0    1    0
```

```
   0    0    1

Matrix Result is: 3X3

   1    3    4

   2    5    7

   5    9    6
```

Q2: Select a number randomly with probability proportional to its magnitude from the given array of n elements

consider an experiment, selecting an element from the list A randomly with probability proportional to its magnitude. assume we are doing the same experiment for 100 times with replacement, in each experiment you will print a number that is selected randomly from A.

```
Ex 1: A = [0 5 27 6 13 28 100 45 10 79]
let f(x) denote the number of times x getting selected in 100 experiments.
f(100) > f(79) > f(45) > f(28) > f(27) > f(13) > f(10) > f(6) > f(5) > f(0)
```

In [45]:
```
from random import uniform
# write your python code here
# you can take the above example as sample input for your program to tes
t
# it should work for any general input try not to hard code for only giv
en input examples

def normalization(A):
    s = sum(A)
    da = []
    for i in A:
        da.append(i/s)
    return da

def cumulative_normalization(A):
    tilda = []
    sum = 0
    for i in A:
        sum += i
        tilda.append(sum)
    return tilda


# you can free to change all these codes/structure
def pick_a_number_from_list(A):
    NORM = normalization(A)
    CNA = cumulative_normalization(NORM)
    # your code here for picking an element from with the probability pr
opotional to its magnitude

    for i in range(len(A)):
        r = uniform(0, 1)
        for j in range(len(A)):
            if r <= CNA[j]:
                return A[j]

def sampling_based_on_magnitued():
    A = [0, 5, 27, 6, 13, 28, 100, 45, 10, 79]
    selected_num = []
    for i in range(1,100):
        number = pick_a_number_from_list(A)
        selected_num.append(number)
    print(selected_num)

sampling_based_on_magnitued()
```

```
[100, 79, 100, 100, 100, 100, 100, 5, 100, 6, 100, 45, 100, 100, 100, 7
9, 79, 28, 13, 100, 100, 6, 45, 28, 79, 79, 28, 79, 79, 79, 100, 28, 4
5, 45, 13, 79, 100, 100, 27, 27, 100, 27, 79, 79, 45, 100, 6, 28, 45, 7
9, 100, 100, 100, 100, 100, 6, 79, 45, 100, 45, 45, 79, 45, 100, 100, 4
5, 100, 100, 27, 100, 100, 28, 27, 100, 79, 100, 100, 27, 79, 5, 45, 10
0, 45, 28, 45, 79, 28, 79, 100, 5, 79, 28, 13, 27, 28, 79, 45, 45, 13]
```

Q3: Replace the digits in the string with

Consider a string that will have digits in that, we need to remove all the characters which are not digits and replace the digits with #

```
Ex 1: A = 234                Output: ###
Ex 2: A = a2b3c4             Output: ###
Ex 3: A = abc                Output:   (empty string)
Ex 5: A = #2a$#b%c%561#      Output: ####
```

In [46]:
```
import re
# write your python code here
# you can take the above example as sample input for your program to tes
t
# it should work for any general input try not to hard code for only giv
en input examples

# you can free to change all these codes/structure
# String: it will be the input to your program

def replace_digits(String):
    # write your code
    #
    s = re.sub('\D', "", String)
#     s = re.sub(r'[a-zA-Z !@$#%^&*()_-{}[]|;:]', "", String)
    c = re.sub(r'[0-9]', "#", s)
    return c # modified string which is after replacing the # with digit
s

String = input() or '#2a$#b%c%561#'
replace_digits(String)
```

```
#2a$#b%c%561#
```

Out[46]:
```
'####'
```

Q4: Students marks dashboard

Consider the marks list of class students given in two lists
Students =
['student1','student2','student3','student4','student5','student6','student7','student8','student9','student10']
Marks = [45, 78, 12, 14, 48, 43, 45, 98, 35, 80]
from the above two lists the Student[0] got Marks[0], Student[1] got Marks[1] and so on.

Your task is to print the name of students

a. Who got top 5 ranks, in the descending order of marks
b. Who got least 5 ranks, in the increasing order of marks
d. Who got marks between >25th percentile <75th percentile, in the increasing order of marks.

```
Ex 1:
Students=['student1','student2','student3','student4','student5','student
6','student7','student8','student9','student10']
Marks = [45, 78, 12, 14, 48, 43, 47, 98, 35, 80]

a.
student8  98
student10 80
student2  78
student5  48
student7  47

b.
student3 12
student4 14
student9 35
student6 43
student1 45

c.
student9 35
student6 43
student1 45
student7 47
student5 48
```

In [47]:
```
# write your python code here
# you can take the above example as sample input for your program to tes
t
# it should work for any general input try not to hard code for only giv
en input examples
students = ['student1', 'student2', 'student3', 'student4', 'student5',
            'student6', 'student7', 'student8', 'student9', 'student10']
marks = [45, 78, 12, 14, 48, 43, 47, 98, 35, 80]


def sort_dict(d, reverse=False):
    """
    Function sort_dict
    This function is used to reverse dictionary based on reverse flag.
    :param d:
      A d that contains dict data.
    :param reverse:
      A reverse that contains boolean value True or False. Default is Fa
lse.
    :return:
      Return sorted dictionary
    """

    return sorted(d.items(), key=lambda i: i[1], reverse=reverse)

# you can free to change all these codes/structure
def display_dash_board(students, marks):
    """
    Function display_dash_board
    This function is used to display students based on condition as give
n.

    :param students:
      A students that contains list of students name.
    :param marks:
      A marks that contains list of students marks.

    :return:
      Return a tuple that contains top 5 students, least 5 students and
      marks between 25 and 75.
    """
    student_dict = dict(zip(students, marks))
    d = sort_dict(student_dict, True)
    # write code for computing top top 5 students
    top_5_students = dict(d[:5])
    # write code for computing top least 5 students
    least_5_students = dict(d[:4:-1])
    # write code for computing top least 5 students
    students_within_25_and_75 = dict(sort_dict({
        k: v for k, v in student_dict.items() if 25 < v < 75
    }))

    return top_5_students, least_5_students, students_within_25_and_75

def display(d):
    """
```

```
    Function display
    This function is used to display dictionary in one line key-value fo
rmat.

    :param d:
      A d that contains  dictionary.

    :return: None
    """
    if not isinstance(d, dict):
        return
    for k, v in d.items():
        print("{} {}".format(k, v))

top_5_students, least_5_students, students_within_25_and_75 = display_da
sh_board(students, marks)
print(" \na.")
display(top_5_students)
print(" \nb.")
display(least_5_students)
print(" \nc.")
display(students_within_25_and_75)
```

```
a.
student8 98
student10 80
student2 78
student5 48
student7 47

b.
student3 12
student4 14
student9 35
student6 43
student1 45

c.
student9 35
student6 43
student1 45
student7 47
student5 48
```

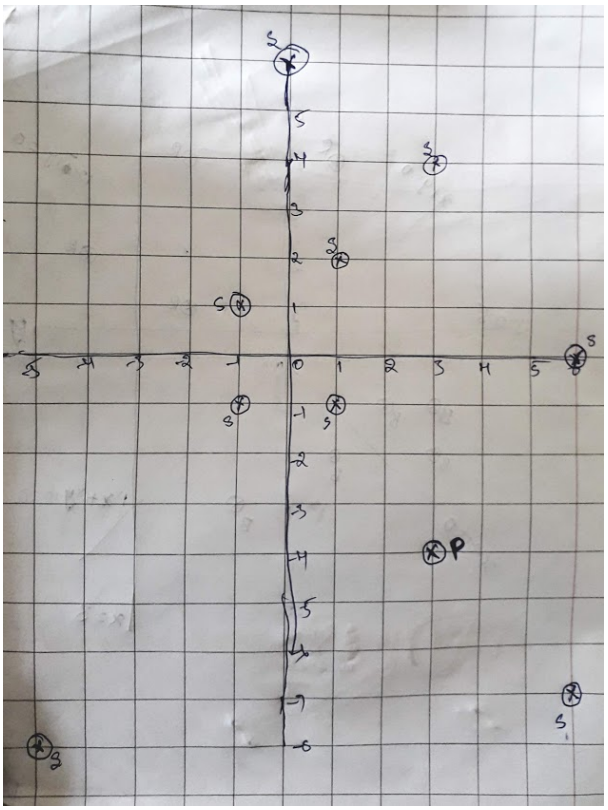
Q5: Find the closest points

Consider you are given n data points in the form of list of tuples like S=[(x1,y1),(x2,y2),(x3,y3),(x4,y4),(x5,y5),.., (xn,yn)] and a point P=(p,q)
your task is to find 5 closest points(based on cosine distance) in S from P

Cosine distance between two points (x,y) and (p,q) is defined as $cos^{-1}(\frac{(x\cdot p+y\cdot q)}{\sqrt{(x^2+y^2)}\cdot\sqrt{(p^2+q^2)}})$

```
Ex:

S= [(1,2),(3,4),(-1,1),(6,-7),(0, 6),(-5,-8),(-1,-1)(6,0),(1,-1)]
P= (3,-4)
```




```
Output:
(6,-7)
(1,-1)
(6,0)
(-5,-8)
(-1,-1)
```

In [48]:
```
import math

# write your python code here
# you can take the above example as sample input for your program to tes
t
# it should work for any general input try not to hard code for only giv
en input examples
# you can free to change all these codes/structure


# here S is list of tuples and P is a tuple ot len=2
def closest_points_to_p(S, P):
    S.sort(key=lambda x: math.acos((x[0]*P[0] + x[1]*P[1]) /(math.sqrt(x
[

0]**2 + x[1]**2) * math.sqrt(P[

0]**2 + P[1]**2))))
    return S[:5]  # its list of tuples

S= [(1,2),(3,4),(-1,1),(6,-7),(0, 6),(-5,-8),(-1,-1),(6,0),(1,-1)]
P= (3,-4)
points = closest_points_to_p(S, P)
print(points) #print the returned values
```

```
[(6, -7), (1, -1), (6, 0), (-5, -8), (-1, -1)]
```

Q6: Find which line separates oranges and apples

Consider you are given two set of data points in the form of list of tuples like

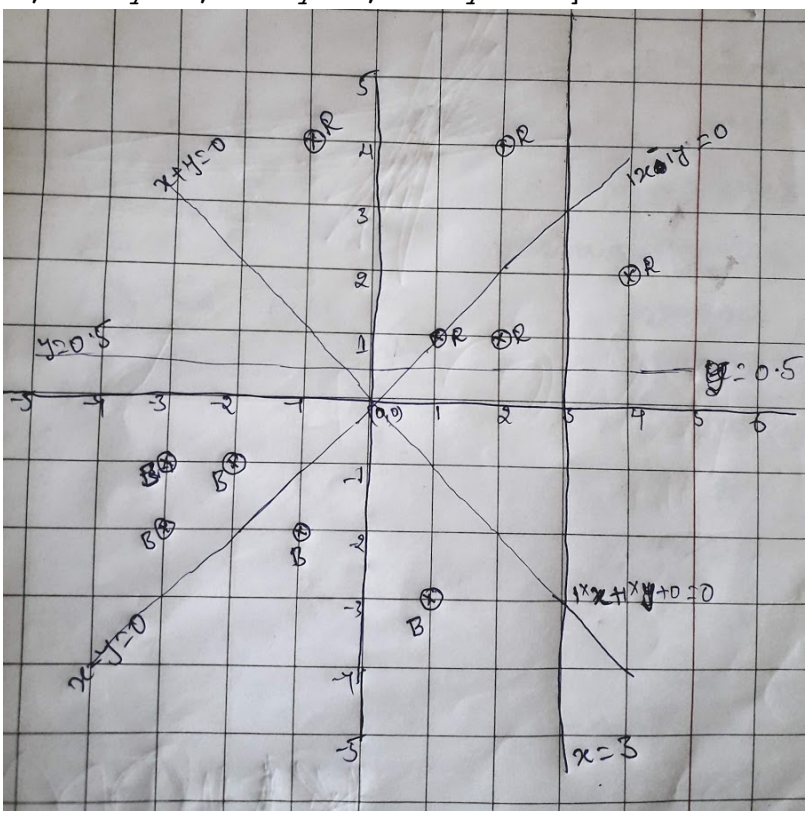
```
Red =[(R11,R12),(R21,R22),(R31,R32),(R41,R42),(R51,R52),..,(Rn1,Rn2)]
Blue=[(B11,B12),(B21,B22),(B31,B32),(B41,B42),(B51,B52),..,(Bm1,Bm2)]
```

and set of line equations(in the string format, i.e list of strings)

```
Lines = [a1x+b1y+c1,a2x+b2y+c2,a3x+b3y+c3,a4x+b4y+c4,..,K lines]
Note: You need to do string parsing here and get the coefficients of x,y and
intercept.
```

Your task here is to print "YES"/"NO" for each line given. You should print YES, if all the red points are one side of the line and blue points are on other side of the line, otherwise you should print NO.

```
Ex:
Red= [(1,1),(2,1),(4,2),(2,4), (-1,4)]
Blue= [(-2,-1),(-1,-2),(-3,-2),(-3,-1),(1,-3)]
Lines=["1x+1y+0","1x-1y+0","1x+0y-3","0x+1y-0.5"]
```




```
Output:
YES
NO
NO
YES
```

In [49]:
```
import math, re
# write your python code here
# you can take the above example as sample input for your program to tes
t
# it should work for any general input try not to hard code for only giv
en input strings


# you can free to change all these codes/structure
def i_am_the_one(red,blue,line):
    a, b, c = [float(coef.strip()) for coef in re.findall(r'[\d\.\-\+]+'
, line)]
    for r in red:
        p = a * r[0] + b * r[1] + c
        if p > 0:
            pass
        else:
            return 'NO'
    for bl in blue:
        p = a * bl[0] + b * bl[1] + c
        if p < 0:
            pass
        else:
            return 'NO'
    return 'YES'

#     Second method
#     for r in red:
#         equation = line.replace('x', '*' + str(r[0])).replace('y',
#                                                               '*' + st
r(r[1]))
#         result = eval(equation)
#         # print(equation)
#         if result > 0:
#             pass
#         else:
#             return "NO"
#     for b in blue:
#         equation = line.replace('x', '*' + str(b[0])).replace('y',
#                                                               '*' + st
r(b[1]))
#         # print(equation)
#         result = eval(equation)
#         if result < 0:
#             pass
#         else:
#             return "NO"
#     return "Yes"


if __name__ == "__main__":
    Red = [(1, 1), (2, 1), (4, 2), (2, 4), (-1, 4)]
    Blue = [(-2, -1), (-1, -2), (-3, -2), (-3, -1), (1, -3)]
    Lines = ["1x+1y+0", "1x-1y+0", "1x+0y-3", "0x+1y-0.5"]
    for i in Lines:
```

```
        yes_or_no = i_am_the_one(Red, Blue, i)
        print(yes_or_no)  # the returned value
```

```
YES
NO
NO
YES
```

Q7: Filling the missing values in the specified format

You will be given a string with digits and '_'(missing value) symbols you have to replace the '_' symbols as explained

```
Ex 1: _, _, _, 24 ==> 24/4, 24/4, 24/4, 24/4 i.e we. have distributed the 24
equally to all 4 places

Ex 2: 40, _, _, _, 60 ==> (60+40)/5,(60+40)/5,(60+40)/5,(60+40)/5,(60+40)/5
 ==> 20, 20, 20, 20, 20 i.e. the sum of (60+40) is distributed qually to all
5 places

Ex 3: 80, _, _, _, _  ==> 80/5,80/5,80/5,80/5,80/5 ==> 16, 16, 16, 16, 16 i.
e. the 80 is distributed qually to all 5 missing values that are right to it

Ex 4: _, _, 30, _, _, _, 50, _, _
==> we will fill the missing values from left to right
    a. first we will distribute the 30 to left two missing values (10, 10, 1
0, _, _, _, 50, _, _)
    b. now distribute the sum (10+50) missing values in between (10, 10, 12,
12, 12, 12, 12, _, _)
    c. now we will distribute 12 to right side missing values (10, 10, 12, 1
2, 12, 12, 4, 4, 4)
```

for a given string with comma seprate values, which will have both missing values numbers like ex: "_, _, x, _, _, _" you need fill the missing values Q: your program reads a string like ex: "_, _, x, _, _, _" and returns the filled sequence Ex:

```
Input1: "_,_,_,24"
Output1: 6,6,6,6

Input2: "40,_,_,_,60"
Output2: 20,20,20,20,20

Input3: "80,_,_,_,_"
Output3: 16,16,16,16,16

Input4: "_,_,30,_,_,_,50,_,_"
Output4: 10,10,12,12,12,12,4,4,4
```

In [50]:
```
# write your python code here
# you can take the above example as sample input for your program to tes
t
# it should work for any general input try not to hard code for only giv
en input strings


# you can free to change all these codes/structure
def fill_values(next_val, s, root, number1, number2, split):
    s += 1
    if root == 1:
        pass
    else:
        for i in range(next_val, s):
            split[i] = int((number1 + number2) / root)


def curve_smoothing(S):
    split = S.strip().split(",")
    l = 0
    r = len(split)
    count = 1
    next_val = 0

    for i in range(r):
        if split[i] == "_":
            split[i] = 0
        else:
            split[i] = int(split[i])
    while l < r:
        if split[l] != 0:
            root = l - next_val + 1
            fill_values(next_val, l, root, split[next_val], split[l], sp
lit)
            next_val = l
        if count == r:
            if split[l] == 0:
                root = (r - 1) - next_val + 1
                fill_values(next_val, r - 1, root, split[next_val],
                            split[r - 1], split)
        l += 1
        count += 1
    return split


S=  "_,_,30,_,_,_,50,_,_"
smoothed_values= curve_smoothing(S)
print(smoothed_values)
```

```
[10, 10, 12, 12, 12, 12, 4, 4, 4]
```

Q8: Find the probabilities

You will be given a list of lists, each sublist will be of length 2 i.e. [[x,y],[p,q],[l,m]..[r,s]] consider its like a martrix of n rows and two columns

1. The first column F will contain only 5 uniques values (F1, F2, F3, F4, F5)
2. The second column S will contain only 3 uniques values (S1, S2, S3)

```
your task is to find
a. Probability of P(F=F1|S==S1), P(F=F1|S==S2), P(F=F1|S==S3)
b. Probability of P(F=F2|S==S1), P(F=F2|S==S2), P(F=F2|S==S3)
c. Probability of P(F=F3|S==S1), P(F=F3|S==S2), P(F=F3|S==S3)
d. Probability of P(F=F4|S==S1), P(F=F4|S==S2), P(F=F4|S==S3)
e. Probability of P(F=F5|S==S1), P(F=F5|S==S2), P(F=F5|S==S3)
```

Ex:

```
[[F1,S1],[F2,S2],[F3,S3],[F1,S2],[F2,S3],[F3,S2],[F2,S1],[F4,S1],[F4,S3],[F
5,S1]]

a. P(F=F1|S==S1)=1/4, P(F=F1|S==S2)=1/3, P(F=F1|S==S3)=0/3
b. P(F=F2|S==S1)=1/4, P(F=F2|S==S2)=1/3, P(F=F2|S==S3)=1/3
c. P(F=F3|S==S1)=0/4, P(F=F3|S==S2)=1/3, P(F=F3|S==S3)=1/3
d. P(F=F4|S==S1)=1/4, P(F=F4|S==S2)=0/3, P(F=F4|S==S3)=1/3
e. P(F=F5|S==S1)=1/4, P(F=F5|S==S2)=0/3, P(F=F5|S==S3)=0/3
```

In [51]:
```
# write your python code here
# you can take the above example as sample input for your program to tes
t
# it should work for any general input try not to hard code for only giv
en input strings



# you can free to change all these codes/structure


def compute_conditional_probabilites(A):
    """
    Function compute_conditional_probabilites
    This function is used to find conditional probabilites

    :param A:
      An A that contains matrix or list of list data

    :return:
      Return tuple of dict.
    """
    S = dict()
    FS = dict()
    for k, v in A:
        c1 = 1
        c2 = 1
        if v in S:
            c1 = S[v]
            c1 += 1
        S.update({v: c1})
        kv = k + v
        if kv in FS:
            c2 = FS[kv]
            c2 += 1
        FS.update({k + v: c2})
    return S, FS


def probabilites(S, FS, f, s):
    """
    Function probabilites
    This function is used to compute probability.

    :param S:
      A S that contains dict values.
    :param FS:
      A FS that contains dict values.
    :param f:
      A f that contains string value.
    :param s:
      A s that contains string value.

    :return:
      Return probability value if f and s exist else return 0.
    """
```

```
    return str(FS[f + s]) + '/' + str(S[s]) if f + s in FS and s in S el
se 0


A = [
        ["F1", "S1"],
        ["F2", "S2"],
        ["F3", "S3"],
        ["F1", "S2"],
        ["F2", "S3"],
        ["F3", "S2"],
        ["F2", "S1"],
        ["F4", "S1"],
        ["F4", "S3"],
        ["F5", "S1"]
    ]
S, FS = compute_conditional_probabilites(A)
f = input().capitalize()
s = input().capitalize()
print("P(F={}|S=={}) = {}".format(f, s, probabilites(S, FS, f, s)))
```

```
F1
S1
P(F=F1|S==S1) = 1/4
```

Q9: Operations on sentences

You will be given two sentances S1, S2 your task is to find

```
a. Number of common words between S1, S2
b. Words in S1 but not in S2
c. Words in S2 but not in S1
```

Ex:

```
S1= "the first column F will contain only 5 unique values"
S2= "the second column S will contain only 3 unique values"
Output:
a. 7
b. ['first','F','5']
c. ['second','S','3']
```

In [52]:
```
# write your python code here
# you can take the above example as sample input for your program to tes
t
# it should work for any general input try not to hard code for only giv
en input strings

# you can free to change all these codes/structure
def string_features(S1, S2):
    # your code
    s1 = set(S1.split())
    s2 = set(S2.split())
    a = len(s1 & s2)
    b = s1 - s2
    c = s2 - s1
    return a, b, c

S1= "the first column F will contain only 5 uniques values"
S2= "the second column S will contain only 3 uniques values"
a,b,c = string_features(S1, S2)
print(a)
print(b)
print(c)
```

```
7
{'5', 'F', 'first'}
{'second', '3', 'S'}
```

Q10: Error Function

You will be given a list of lists, each sublist will be of length 2 i.e. [[x,y],[p,q],[l,m]..[r,s]] consider its like a martrix of n rows and two columns

a. the first column Y will contain interger values
b. the second column Y_{score} will be having float values
Your task is to find the value of
$f(Y, Y_{score}) = -1 * \frac{1}{n}\Sigma_{foreachY,Y_{score}pair}(Ylog10(Y_{score}) + (1-Y)log10(1-Y_{score}))$ here n is the number of rows in the matrix

```
Ex:
[[1, 0.4], [0, 0.5], [0, 0.9], [0, 0.3], [0, 0.6], [1, 0.1], [1, 0.9], [1,
0.8]]
output:
0.44982
```

$\frac{-1}{8} \cdot ((1 \cdot log_{10}(0.4) + 0 \cdot log_{10}(0.6)) + (0 \cdot log_{10}(0.5) + 1 \cdot log_{10}(0.5)) + \ldots + (1 \cdot log_{10}(0.8) + 0 \cdot log_{10}(0.2$

In [53]:
```
# write your python code here
# you can take the above example as sample input for your program to tes
t
# it should work for any general input try not to hard code for only giv
en input strings

import math

# -1/8*((1*log10(0.4)+0*log10(0.6))+(0*log10(0.5)+1*log10(0.5))+...+(
# 1*log10(0.8)+0*log10(0.2)))
# you can free to change all these codes/structure


def compute_log_loss(A):
    n = len(A)
    fr = 0
    for y, y_score in A:
        fr += (y * math.log10(y_score) + (1 - y) * math.log10(1 - y_scor
e))
    loss = -1 / n * fr
    return loss

A = [[1, 0.4], [0, 0.5], [0, 0.9], [0, 0.3], [0, 0.6], [1, 0.1], [1, 0.9
],
     [1, 0.8]]
loss = compute_log_loss(A)
print('{0:.5f}'.format(loss))
```

```
0.42431
```